

AsyncPlate - Project Documentation

Table of Contents

1. Problem
 2. Solution
 3. Functional Requirements
 4. Non Functional Requirements
 5. ERD
 6. Technologies and GitHub Link
-

1. Problem

Restaurants often rely on multiple disconnected systems and manual processes to manage daily operations. This creates several challenges:

- Delays in processing customer orders.
- Difficulty tracking inventory levels.
- Unexpected stock shortages.
- Inefficient communication with suppliers.
- Time-consuming manual administrative tasks.
- Lack of real-time updates between customers, chefs, and administrators.
- Difficulty managing promotional offers and customer engagement.

These challenges increase operational costs, reduce efficiency, and negatively impact customer satisfaction.

2. Solution

AsyncPlate is a cloud-based restaurant management system designed to automate and centralize restaurant operations.

The system provides a single platform that allows restaurants to:

- Manage orders in real time.
- Monitor inventories automatically.

- Notify suppliers about low-stock items.
- Send daily inventory requirement reports.
- Manage menus, products, categories, and recipes.
- Create promotional offers.
- Engage customers through notifications, offers .
- Improve communication between all system roles.

The system aims to reduce manual work, improve efficiency, and provide a scalable solution for restaurant management.

3. Functional Requirements

1. Authentication & Authorization

- The system shall allow users to register and create accounts based on predefined roles (Customer, Chef, Administrator).
- The system shall authenticate users securely using email and password.
- The system shall issue JWT tokens upon successful authentication.
- The system shall enforce role-based access control to restrict access to authorized features only.
- The system shall allow users to reset their password in case they forget it through a secure email-based verification process.

2. Menu Management

- The system shall allow administrators to create, update, and delete menu categories.
- The system shall allow administrators to create, update, and delete menu items.
- The system shall allow chefs to manage recipes associated with menu items.
- The system shall allow customers to browse the restaurant menu.
- The system shall allow customers to view detailed information about menu items.

3. Order Management

- The system shall allow customers to create orders.
- The system shall allow customers to track order status in real time.
- The system shall allow customers to view order history.
- The system shall manage the full lifecycle of an order from creation to completion.
- The system shall define and enforce order statuses (Pending, Preparing, Completed, Cancelled).
- The system shall broadcast new orders to chefs in real time.
- The system shall update customers in real time when order statuses change.

4. Chef Management

- The system shall allow chefs to view incoming orders.
- The system shall notify chefs when new orders are placed.
- The system shall allow chefs to update order statuses.
- The system shall allow chefs to view order details and associated recipes.

5. Inventory Management

- The system shall track current stock levels for all inventory items.
- The system shall define minimum stock thresholds for inventory items.
- The system shall automatically deduct inventory quantities when orders are completed.
- The system shall automatically detect low-stock items.
- The system shall generate alerts when inventory falls below minimum levels.

6. Supplier Management

- The system shall associate inventory items with suppliers.
- The system shall generate daily inventory requirement reports.
- The system shall send inventory requirement reports to suppliers via email.

7. Notification Management

- The system shall send notifications for order creation, updates, and completion.
- The system shall send notifications for promotional offers.
- The system shall allow users to view their notifications.
- The system shall allow users to mark notifications as read.

8. Offer Management

- The system shall allow administrators to create promotional offers.
- The system shall distribute promotional offers to VIP customers.
- The system shall notify VIP customers when new offers become available.

9. Review Management

- The system shall allow customers to submit ratings and reviews for completed orders.
- The system shall allow customers to view their previous reviews.

4. Non-Functional Requirements

1. Performance Requirements

- The system shall respond to user requests within an acceptable response time.
- The system shall deliver real-time notifications with minimal latency.
- The system shall support concurrent users without significant performance degradation.

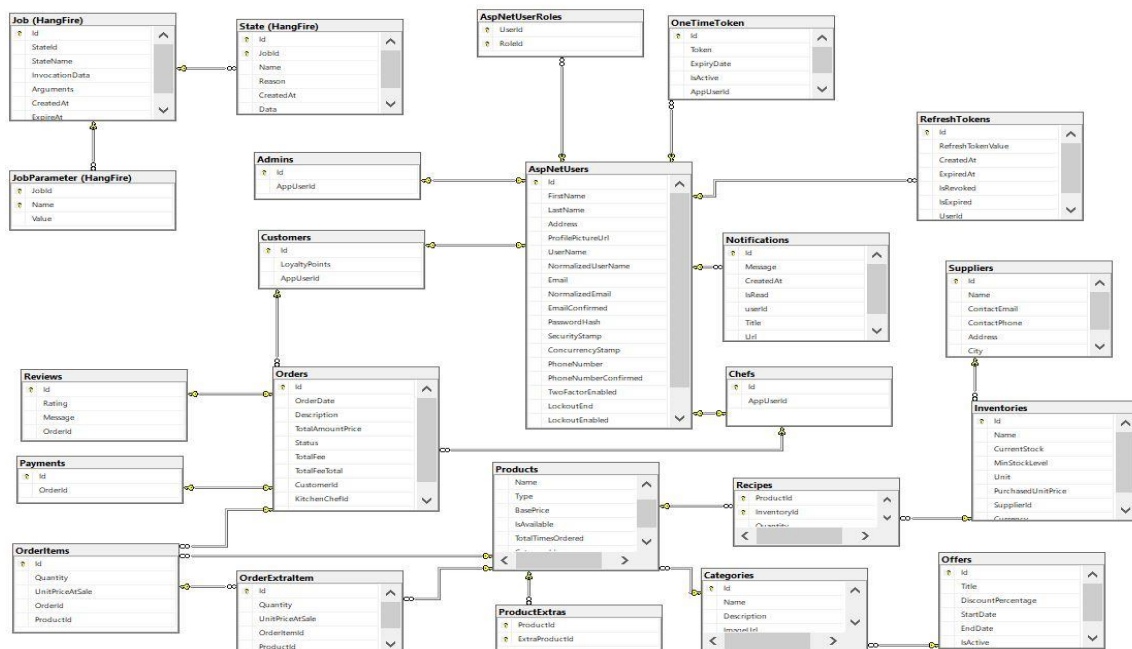
2. Security Requirements

- The system shall store user passwords using a secure hashing algorithm.
- The system shall protect all authentication and authorization operations using HTTPS.
- The system shall secure protected API endpoints using JWT authentication.
- The system shall enforce role-based access control to prevent unauthorized access.
- The system shall generate password reset tokens that expire after a predefined period.
- The system shall prevent access to resources without a valid authentication token.

3. Maintainability Requirements

- The system shall follow Clean Architecture principles.

5. ERD



6. Technologies and GitHub Link

Backend	ASP.NET Core – Web API - C# - SQL server – Hangfire – SignalR – Jwt
Architecture	Clean Architecture - Repository Pattern – Unit of work – Dependency injection
Tools	Postman – Swagger – git - github
Github	https://github.com/alyaaa7med/AsyncPlate
